

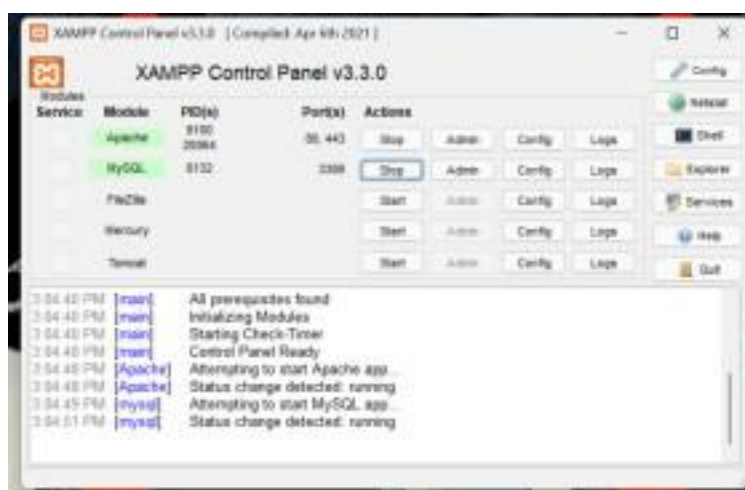
Mendoza, Geraldine C.
Gilhang, Jhulia Reign
Santos, Janier Levine U.
Serad, Sittie Aisah S.

C303

As we download the approved dataset, *Student Depression & Lifestyle* <https://www.kaggle.com/datasets/alidinwhyudii/student-depression-and-lifestyle-100k-data?resource=download>. We will begin the phase for Data Preparation and Data Cleaning using CLEAN Framework.

For the data cleaning and preparation, I decided to use XAMPP and MySQL Workbench instead of Power BI. At first, it really seemed like doing all the cleaning inside Power BI's Power Query would be much easier since our team's data modeling, dashboard, and visualization will have to be built there anyway. However, I chose to use a dedicated SQL database because it will give us a much better setup for our project pipeline. First, I wanted to separate our database backend (MySQL) from our visual frontend (Power BI), which is how actual data professionals organize their pipelines. Second, since our instructor strictly requires us to use the CLEAN framework and document our work to prove we did it ourselves, I knew that by writing out this explicitly SQL queries would give me a clear code log to show as solid proof of my independent work. Lastly, by cleaning all 100,000 rows directly in MySQL, I can make sure the dataset is already pristine before it ever touches Power BI. This saves a lot of computer memory, fixes any loading lag, and makes it way easier for my next group member to jump straight into the data modeling and build the Star Schema relationships without any extra hassle.

1. We begin by opening the XAMPP, launching the XAMPP will allow us to find Apache and MySQL in the XAMPP Control Panel. Click the **Start** button (both for Apache and MySQL). Wait for a few minutes until their background turns green.

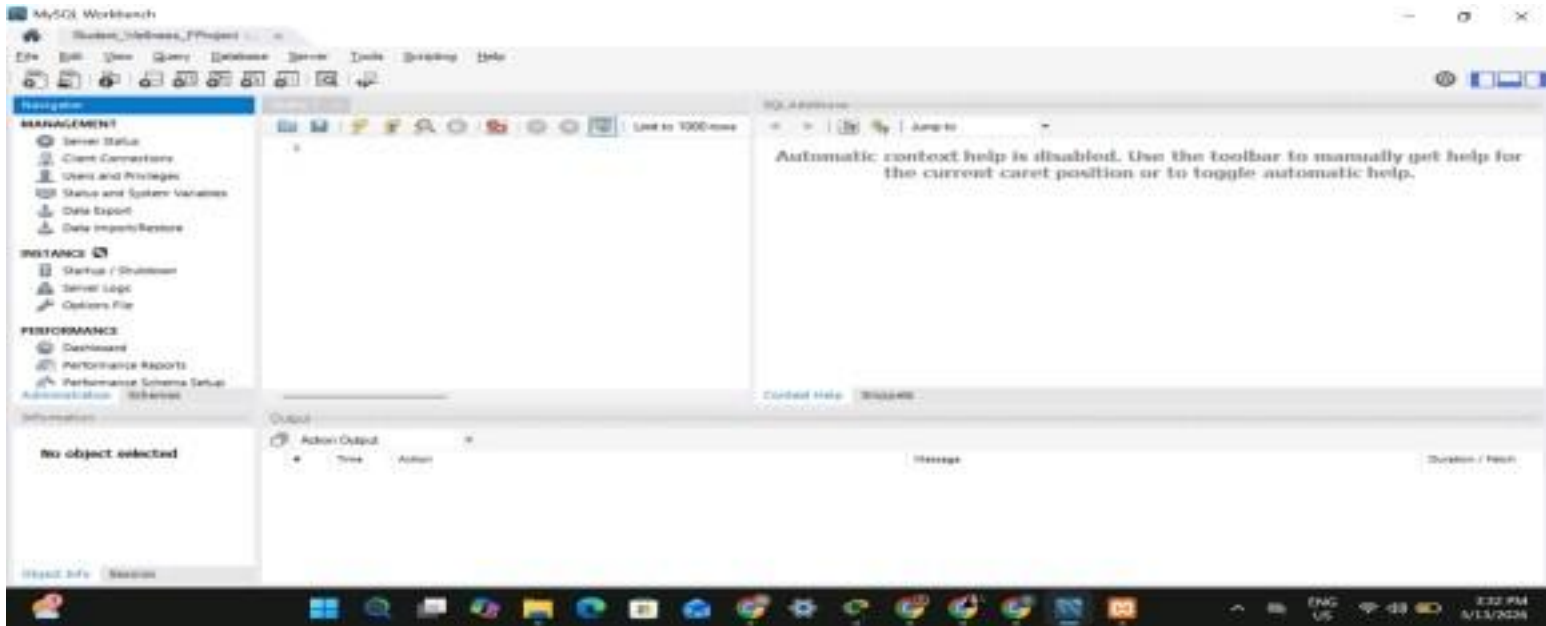


2. Next thing we have to open now is the MySQL Workbench. Once it is launched, we should create a connection by clicking the + icon next to "MySQL connections" near the

top left.

- We also set the connection name to: *Student_Wellness_FProject*
- Hostname to: 127.0.0.1
- Port to: 3306
- Username to: root
- And we will leave the password *blank*

3. We click the **OK** button to save the connection.

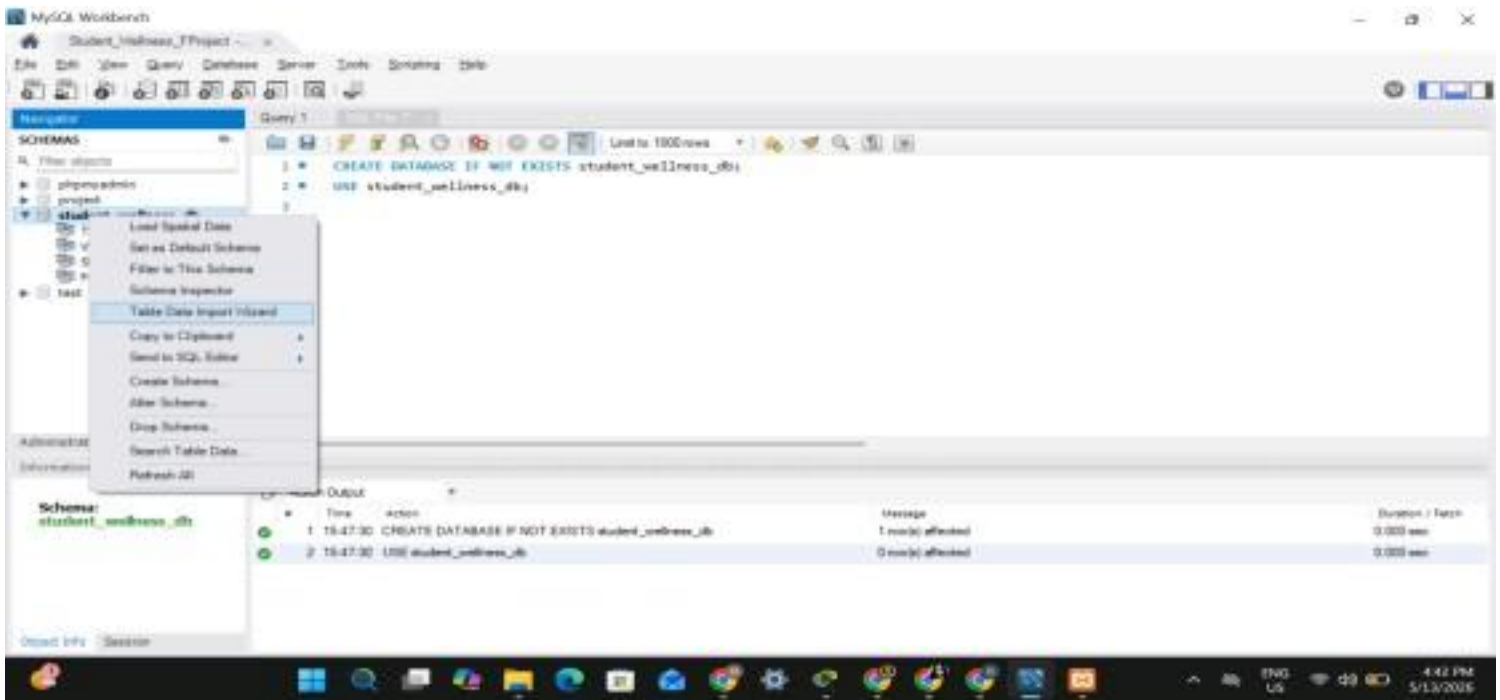


4. Tadaaah! We're in! If we want to remove the unnecessary or sidebar that we didn't need, we could click the word *View* at the top and hover over *Panels* and click the *Hide Secondary Sidebar*

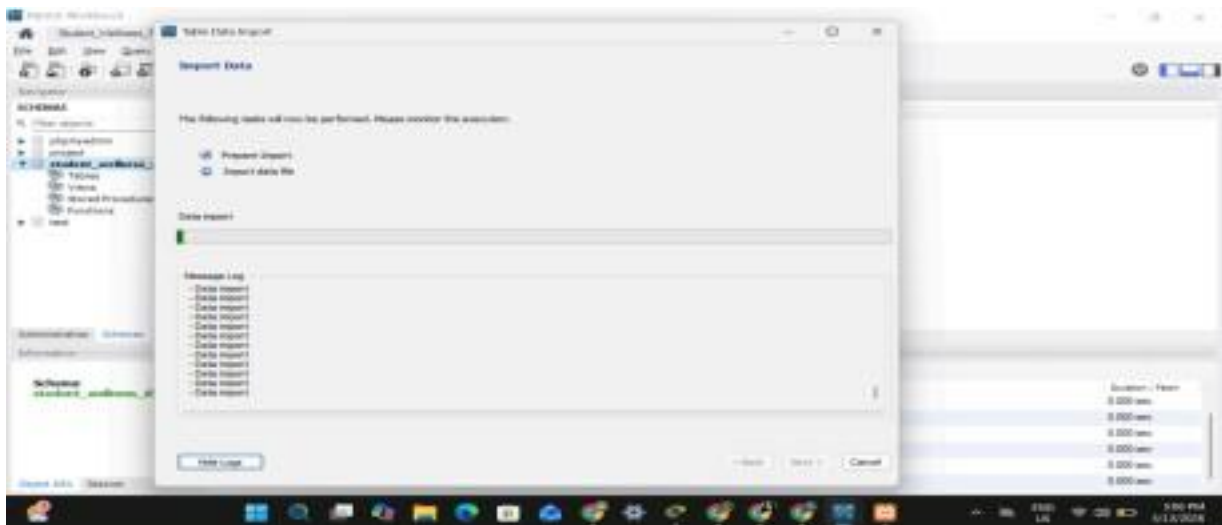


5. We could click the icon **SQL with a +** if we want to create a new query (and that's how we will do it). Now, we will create the environment so we could start the cleaning for our dataset. We start our data cleaning by creating and using the database we just created *student_wellness_db* and run it with the lightning bolt icon at the top (that means it will execute). Refresh it and we will see the *student_wellness_db* appear under the

Schemas tab. Right-click our *student_wellness_db* schema and select the **Table Data Import Wizard**.



6. We will browse the dataset from the source file that we acquired from Kaggle, click **Open** and then click the word *Next > Next* again > yes, *Next* again > hehe, *Next* again until we see that it's starting to import the data. Since, we have a huge dataset, as in huge (*well its 100,000 records*) it will take some time to load or to import our dataset.



7. Yehey! After waiting for so long, we have already imported the dataset that we need to clean. After that, we need to click (again) the refresh indicator near the word **SCHEMAS** for us to see the data we have imported under *Tables*.



8. Moving on, we will open a new query tab to start our data cleaning. We will begin with the execution of **CLEAN Framework**. We begin by C - Conceptualize. In this phase, we run a query to verify that our *student_lifestyle_100k* table contains exactly 100,000 records and that every **Student_ID** is distinct. This will establish our data boundaries and proves that no data was dropped or corrupted during the ingestion process.

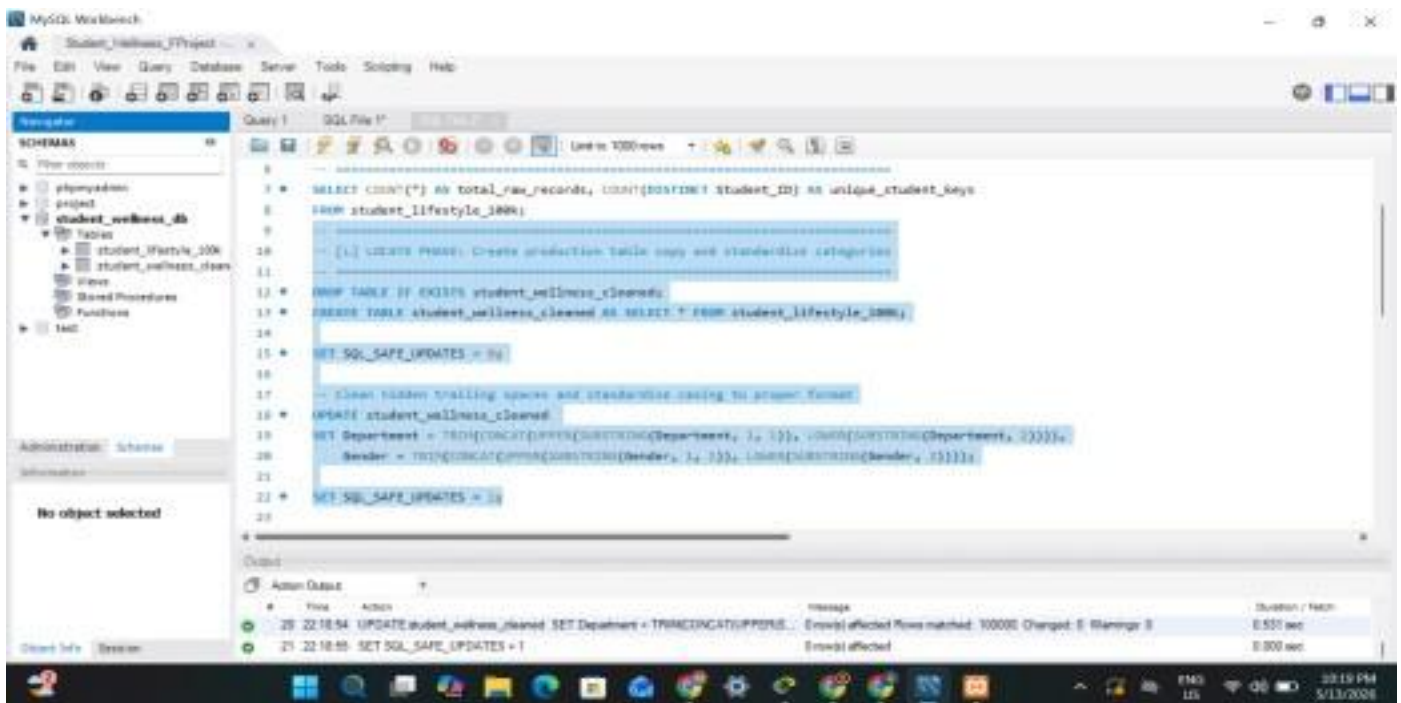
```

1  -- Use our active project schema
2  *  USE student_wellness_db;
3
4  -- =====
5  -- [C] CONCEPTUALIZE PHASE: Establish data boundaries and ingestion counts
6  -- =====
7  *  SELECT COUNT(*) AS total_raw_records, COUNT(DISTINCT Student_ID) AS unique_student_keys
8  FROM student_lifestyle_100k;

```

Result Grid		
	total_raw_records	unique_student_keys
▶	100000	100000

9. Next, we move to L - Locate. In this step, we duplicate our new dataset into a secondary production table named *student_wellness_cleaned* so we do not overwrite our source data. We then run an update query to clean up categorical text columns like **Department** and **Gender**. By applying **TRIM** and casing functions, we will remove any hidden trailing spaces and standardize the text formatting so it does not break our downstream dashboard filters.



10. We then proceed to E - Evaluate. We will perform a health audit on our numerical columns by querying the minimum and maximum boundaries for **Age**, **CGPA**, and **Study Hours**. This check validates that our continuous variables sit within logical real-world human limits (such as CGPA between 0.0 and 4.0) and confirms that the dataset contains no impossible structural outliers.

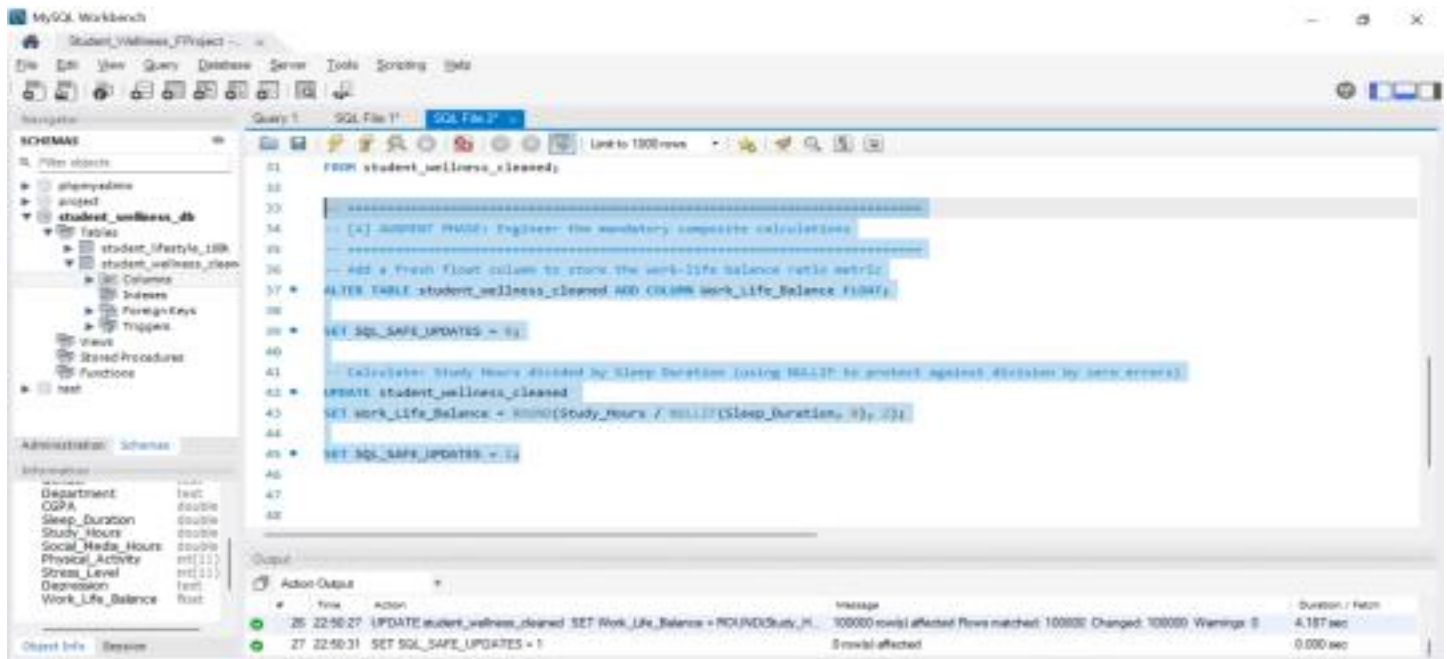
```

-- [E] EVALUATE PHASE: Check numerical ranges for out-of-bounds outliers
SELECT
    MIN(Age) AS youngest, MAX(Age) AS oldest,
    MIN(CGPA) AS worst_gpa, MAX(CGPA) AS best_gpa,
    MIN(Study_Hours) AS min_study_hours, MAX(Study_Hours) AS max_study_hours
FROM student_wellness_cleaned;

```

	youngest	oldest	worst_gpa	best_gpa	min_study_hours	max_study_hours
▶	18	24	1.56	4	0	12.8

11. Moving forward, we will then execute A - Augment to add analytical value to the dataset. As we can see in the picture below, I write an **ALTER TABLE** statement to append a new column called **Work_Life_Balance** to our database schema. We calculate this new metric by dividing a student's **Study_Hours** by their **Sleep_Duration**, rounding it to two decimal places while using a defensive **NULLIF** clause to completely protect our pipeline against division-by-zero database errors.



12. Yehey! Finally, we complete our framework with N -Note. We execute a final **SELECT** query with a **LIMIT**

10 clause to run a quick quality assurance look over newly generated data layers. This final inspection ensures our transformations are executed properly and creates an audit trail, verifying that the data is structured, pristine, and ready for our member who will handle the Data Modeling to import into Power BI.



13. After running our final quality check, we look at the Result Grid at the bottom of the screen. We need to verify that our newly calculated **Work_Life_Balance** column displays perfectly next to the other attributes for all of our 100,000 clean data entries.

Student_ID	Age	Gender	Department	CGPA	Sleep_Duration	Study_Hours	Social_Media_Hours	Physical_Activity	Stress_Level	Depression	Work-Life_Balance
1801	22	Female	Science	3.5	7.3	3.3	3.4	114	3	False	0.45
1802	20	Male	Engineering	3.72	5.5	7.2	6	142	2	False	1.31
1803	20	Male	Medical	3.01	5.4	2.3	1.8	137	3	False	0.45
1804	21	Male	Engineering	3.63	6.1	2	4.8	136	3	False	0.25
1805	19	Male	Arts	3.14	6.8	2.6	4.3	4	6	False	0.38
1806	22	Female	Medical	3.31	6.3	6.8	4.1	128	2	False	0.54
1807	18	Male	Science	2.14	4.4	4	3.1	73	8	True	0.91
1808	18	Male	Engineering	3.27	7	3.4	7.6	68	4	False	0.49
1809	21	Male	Business	2.71	4.8	8.6	0.3	82	3	False	1.79
1810	24	Male	Engineering	3.36	5.7	1.5	3.7	59	3	False	0.26

#	Time	Action	Message	Duration / Fetch
34	02:39:37	SELECT	Student_ID, Age, Gender, Department, CGPA, Sleep, ... 100000 rows returned	0.016 sec / 0.813 sec
35	02:39:44	SELECT	Student_ID, Age, Gender, Department, CGPA, Sleep, ... 100000 rows returned	0.008 sec / 0.793 sec

14. Once everything looks good, we can finally export the dataset! Yey. To do this, we look right above the data results grid pane and click the **Export** button (a small icon that looks like a little spreadsheet grid with a blue arrow pointing outward).

15. A window will pop up asking where we want to save it. We will set the format dropdown menu to CSV, name our final output exactly *student_wellness_cleaned.csv* , and save it straight to our desktop.

16. Yehey! My job as the Data Preparation and Data Cleaning Lead is now officially done. All that is left to do is open our group chat, upload this to my members, and share our documentation so the next member can jump straight into power BI for the Data Modeling Phase.

Modeling Phase

After completing the Data Preparation and Data Cleaning phase, the cleaned dataset *student_wellness_cleaned.csv* was exported from MySQL and prepared for modeling. The next stage of the project focuses on Data Modeling, where the cleaned flat dataset is transformed into a structured Star Schema using Microsoft Power BI.

The primary objective of this phase is to organize the dataset into logical tables consisting of one Fact Table and several Dimension Tables. This structure improves data organization, enhances performance, and allows efficient dashboard creation and analytical reporting within Power BI.

Using Power Query Editor, the dataset was divided into the following tables:

- *student_wellness_cleaned* – Source reference table
- *fact_student_wellness* – Central fact table
- *student_dim* – Student demographic dimension
- *department_dim* – Department lookup dimension
- *behavior_dim* – Student behavior dimension
-

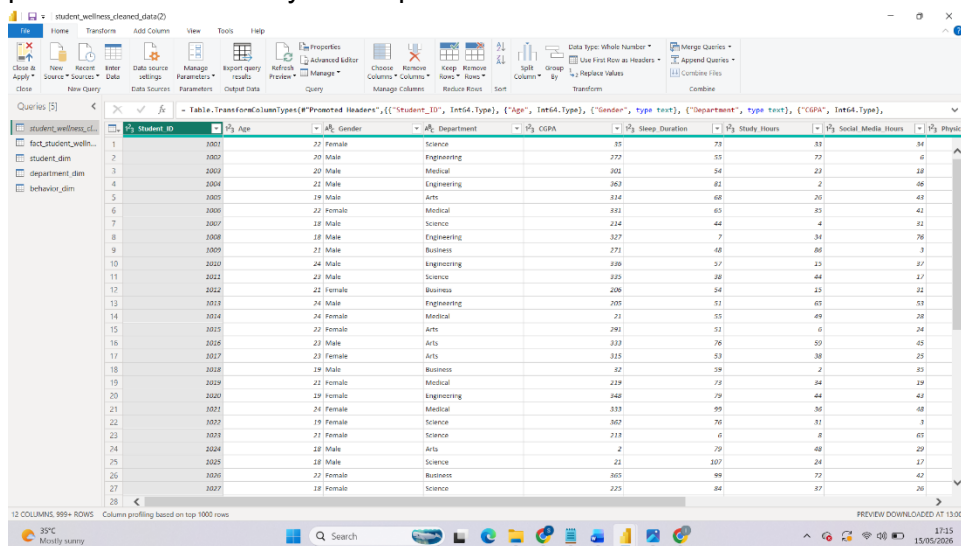
The following sections describe the step-by-step process used to construct the data model.

Step 1: Loading the Cleaned CSV into Power BI

The process begins by opening Power BI Desktop and importing the cleaned dataset. From the Home, Get Data, Text/CSV is selected, and the file student_wellness_cleaned.csv is loaded into Power Query Editor.

Upon loading, all dataset columns become visible, including the engineered Work_Life_Balance column created during the CLEAN Framework phase. This imported table serves as the base reference query, which is used to generate all succeeding dimension and fact tables without modifying the original data.

The successful loading confirms that column headers, data types, and records were correctly preserved from the MySQL export.



Student ID	Age	Gender	Department	CGPA	Sleep Duration	Study Hours	Social Media Hours	Physio
1	22	Female	Science	85	73	53	53	53
2	20	Male	Engineering	772	55	72	6	6
3	20	Male	Medical	305	54	23	28	28
4	22	Male	Engineering	352	81	2	46	46
5	19	Male	Arts	314	68	26	43	43
6	22	Female	Medical	331	65	35	42	42
7	18	Male	Science	234	44	4	31	31
8	18	Male	Engineering	327	7	34	76	76
9	21	Male	Business	271	48	36	3	3
10	24	Male	Engineering	336	37	23	17	17
11	22	Male	Science	335	28	44	17	17
12	22	Female	Business	206	34	23	31	31
13	24	Male	Engineering	205	51	45	53	53
14	24	Female	Medical	71	55	49	28	28
15	22	Female	Arts	240	51	6	24	24
16	23	Male	Arts	333	76	55	45	45
17	23	Female	Arts	335	53	38	25	25
18	19	Male	Business	52	59	2	35	35
19	22	Female	Medical	229	73	34	19	19
20	19	Female	Engineering	348	79	44	43	43
21	24	Female	Medical	313	59	36	48	48
22	19	Female	Science	362	76	31	3	3
23	21	Female	Science	213	6	6	6	6
24	18	Male	Arts	2	79	48	29	29
25	20	Male	Science	21	107	24	17	17
26	22	Female	Business	305	99	72	42	42
27	18	Female	Science	225	84	37	26	26

Step 2: Creating the fact_student_wellness Table

The central component of the Star Schema is the fact_student_wellness table.

A final reference query was created from the original dataset, retaining only analytical measures and foreign keys:

- Student_ID
- dept_id
- behavior_id
- CGPA
- Stress_Level
- Work_Life_Balance
- Depression

The dept_id and behavior_id fields act as foreign keys linking the fact table to their respective dimension tables. Proper data types were verified to ensure compatibility with Power BI relationships.

Queries [5] < X ✓ fx = Table.TransformColumnTypes(#"Removed Columns1",{{"Student_ID", Int64.Type}})

	Student_ID	dept_id	behavior_id	CGPA	Stress_Level	Work_Life_Balance	Depression
1	10001	1	1	3.5	5	0.45	False
2	10002	2	2	2.72	2	1.11	False
3	10003	3	3	3.01	3	0.43	False
4	10004	4	4	3.63	3	0.25	False
5	10005	5	5	3.14	6	0.88	False
6	10006	6	6	3.31	2	0.54	False
7	10007	7	7	2.14	8	0.91	True
8	10008	8	8	3.27	4	0.49	False
9	10009	9	9	2.71	5	1.79	False
10	10010	10	10	3.36	3	0.26	False
11	10011	11	11	3.35	6	1.16	False
12	10012	12	12	2.06	2	0.28	False
13	10013	13	13	2.05	4	1.27	False
14	10014	14	14	2.1	4	0.89	False
15	10015	15	15	2.91	3	0.12	True
16	10016	16	16	3.33	4	0.78	False
17	10017	17	17	3.15	6	0.72	False
18	10018	18	18	3.3	5	0.34	False
19	10019	19	19	2.19	3	0.47	False
20	10020	20	20	3.48	3	0.56	False
21	10021	21	21	3.33	2	0.36	False
22	10022	22	22	3.82	6	0.41	False
23	10023	23	23	2.13	3	0.13	False
24	10024	24	24	2	3	0.61	False
25	10025	25	25	2.1	3	0.22	False
26	10026	26	26	3.85	4	0.73	False
27	10027	27	27	2.25	8	0.44	False
28	10028	28	28	3.17	5	1.61	False

This table contains measurable data used for analysis and visualization.

Step 3: Creating the student_dim Table

The first dimension table created is student_dim, which stores student demographic information. A reference query was created from the original dataset to avoid duplicating data. The query was renamed student_dim, and only relevant attributes were retained:

- Student_ID
- Age
- Gender
- Physical_Activity

All unnecessary columns were removed using Remove Other Columns. Duplicate records were eliminated based on Student_ID to ensure each student appears only once. This step guarantees that the table functions properly as a dimension table containing unique entities.

Queries [5] <

fx = Table.TransformColumnTypes(#"Removed Duplicates",{"Physical_Activity", type text})

	i ₃ Student_ID	i ₃ Age	A _C Gender	A _C Physical_Activity
1	1001	22	Female	114
2	1002	20	Male	142
3	1003	20	Male	137
4	1004	21	Male	130
5	1005	19	Male	4
6	1006	22	Female	129
7	1007	18	Male	73
8	1008	18	Male	68
9	1009	21	Male	82
10	1010	24	Male	59
11	1011	23	Male	95
12	1012	21	Female	127
13	1013	24	Male	107
14	1014	24	Female	47
15	1015	22	Female	72
16	1016	23	Male	53
17	1017	23	Female	24
18	1018	19	Male	145
19	1019	21	Female	31
20	1020	19	Female	129
21	1021	24	Female	117
22	1022	19	Female	17
23	1023	21	Female	116
24	1024	18	Male	7
25	1025	18	Male	133
26	1026	22	Female	132
27	1027	18	Female	70
28	1028	24	Male	142

Step 4: Creating the department_dim Table

Next, the department_dim table was constructed to serve as a lookup table for academic departments.

A reference to the original dataset was again created and renamed department_dim. Only the Department column was retained. An Index Column was added and renamed dept_id, which acts as the primary key.

Duplicate department values were removed to ensure a clean and unique list of departments. This table provides standardized department identifiers that will later connect to the fact table.

Queries [5] <

fx = Table.Distinct(#"Reordered Column")

1 ² dept_id	A ^B Department
1	1 Science
2	2 Engineering
3	3 Medical
4	4 Engineering
5	5 Arts
6	6 Medical
7	7 Science
8	8 Engineering
9	9 Business
10	10 Engineering
11	11 Science
12	12 Business
13	13 Engineering
14	14 Medical
15	15 Arts
16	16 Arts
17	17 Arts
18	18 Business
19	19 Medical
20	20 Engineering
21	21 Medical
22	22 Science
23	23 Science
24	24 Arts
25	25 Science
26	26 Business
27	27 Science
28	28 Engineering

Step 5: Creating the behavior_dim Table

The behavior_dim table represents student lifestyle and behavioral characteristics. From another reference query, only the following columns were retained:

- Sleep_Duration
- Study_Hours
- Social_Media_Hours

An Index Column was added and renamed behavior_id to serve as the table's primary key. Duplicate behavioral records were removed to ensure each behavior profile is uniquely represented.

This dimension enables detailed behavioral analysis while preventing redundancy within the main dataset.

Queries [5] <

fx = Table.Distinct(#"Reordered Columns", {"behavior_id"})

	1 ² ₃ behavior_id	1 ² ₃ Sleep_Duration	1 ² ₃ Study_Hours	1 ² ₃ Social_Media_Hours
1	1	73	33	34
2	2	55	72	6
3	3	54	23	18
4	4	81	2	46
5	5	68	26	43
6	6	65	35	41
7	7	44	4	31
8	8	7	34	76
9	9	48	86	3
10	10	57	15	37
11	11	38	44	17
12	12	54	15	31
13	13	51	65	53
14	14	55	49	28
15	15	51	6	24
16	16	76	59	45
17	17	53	38	25
18	18	59	2	35
19	19	73	34	19
20	20	79	44	43
21	21	99	36	48
22	22	76	31	3
23	23	6	8	65
24	24	79	48	29
25	25	107	24	17
26	26	99	72	42
27	27	84	37	26
28	28	49	79	45

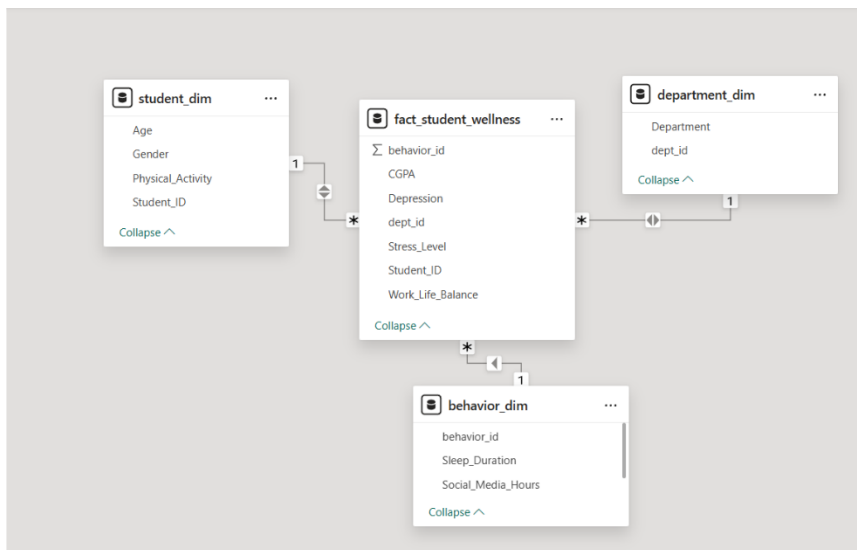
Step 6: Building Star Schema Relationships in Model View

After completing all transformations, the queries were applied and loaded into Power BI. The Model View was then used to establish relationships between tables.

The following relationships were created:

- fact_student_wellness.Student_ID → student_dim.Student_ID (Many-to-One)
- fact_student_wellness.dept_id → department_dim.dept_id (Many-to-One)
- fact_student_wellness.behavior_id → behavior_dim.behavior_id (Many-to-One)

Each relationship follows the standard Star Schema design, where the fact table resides at the center and dimension tables surround it. This structure supports efficient filtering, slicing, and aggregation during dashboard analysis.



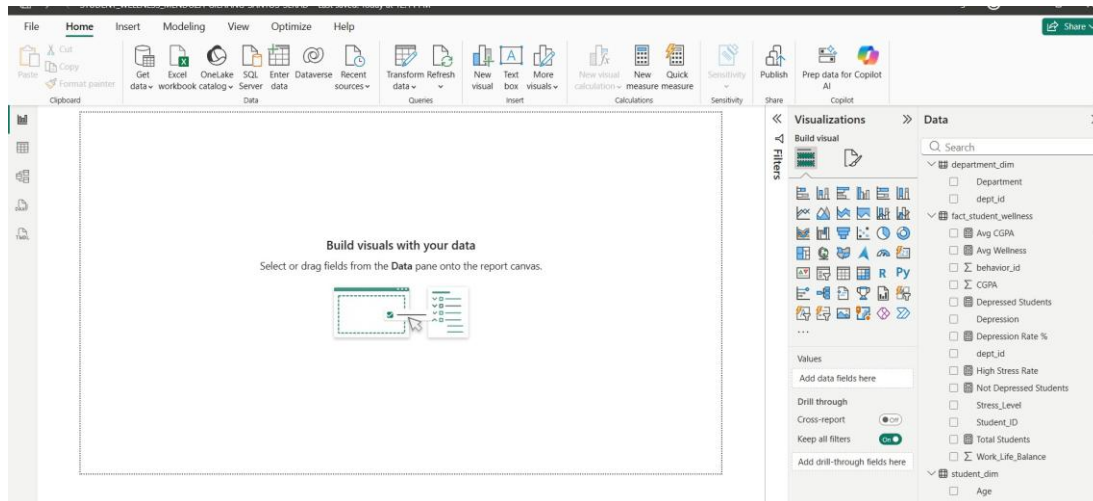
Summary

The Data Modeling phase successfully transformed the flat `student_wellness_cleaned.csv` dataset into a structured Star Schema within Power BI. The model now consists of one fact table connected to three dimension tables through properly defined relationships.

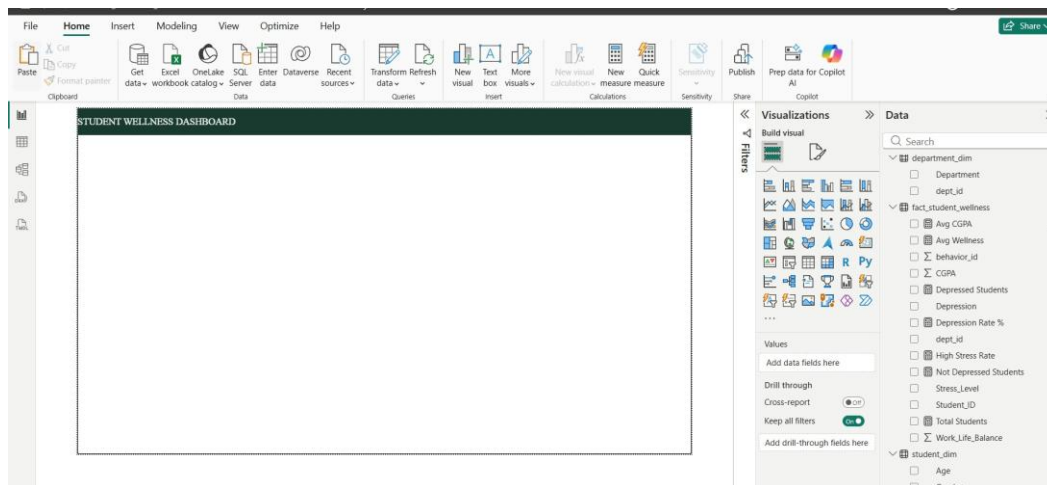
By organizing the dataset into normalized tables, the model improves performance, ensures data consistency, and prepares the project for the Dashboard and Data Visualization Phase. With this foundation completed, the next project member can proceed directly to creating interactive reports and analytical dashboards.

Visualization

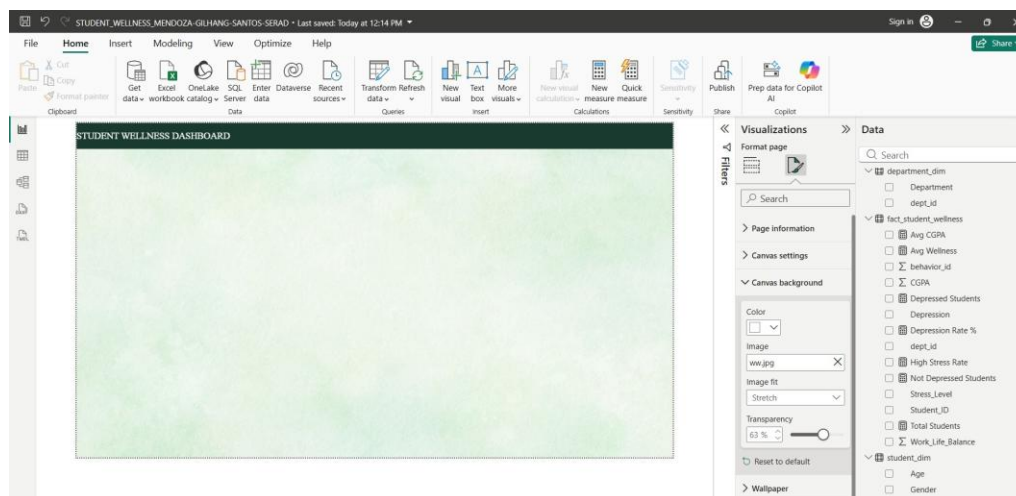
After the Data Preparation and Data Cleaning phase using the CLEAN Framework and the completion of the Data Modeling phase through the implementation of a Star Schema model in Power BI, we now proceed to the final phase of the project which is the Data Visualization and Dashboard Development Phase. In this stage, the cleaned and structured dataset is transformed into a fully interactive analytical dashboard capable of presenting meaningful insights regarding student wellness, stress levels, depression indicators, sleep behavior, and academic performance'



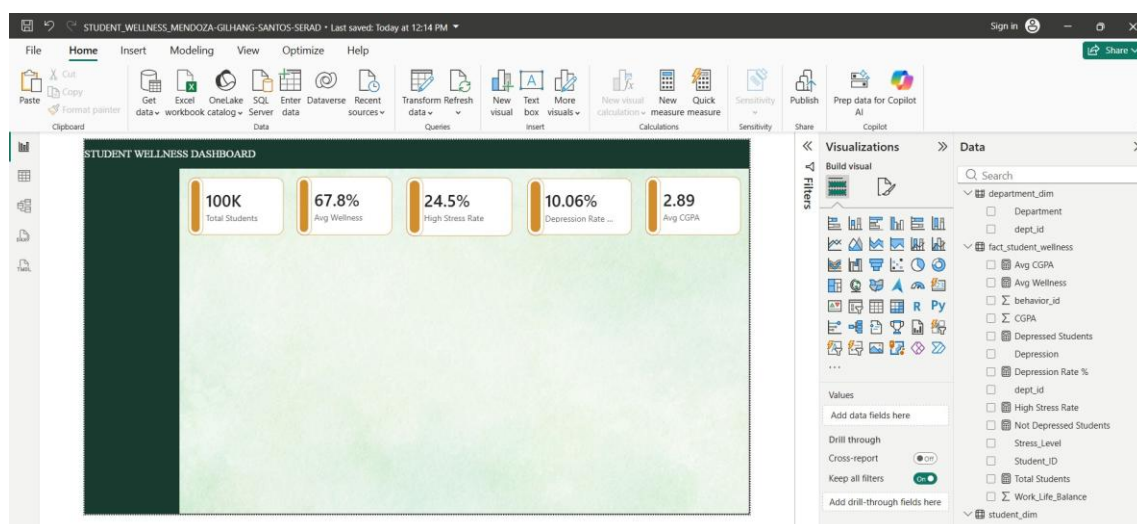
We first design the overall dashboard layout by creating a wellness-themed interface using a dark green and gold color palette. We also insert a title text box labeled:



This title serves as the main header of the visualization system and establishes the dashboard's analytical theme.



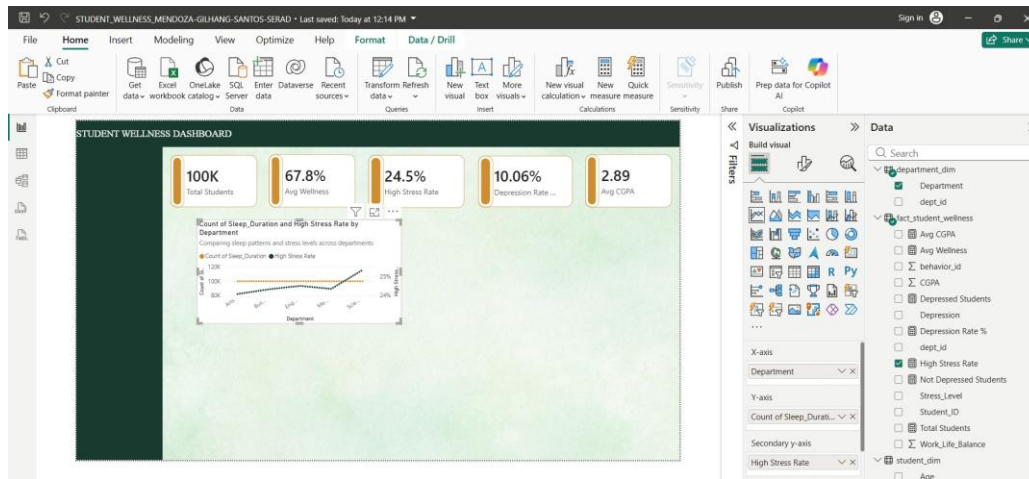
Next, we create KPI Cards positioned horizontally at the top section of the dashboard to summarize the most important wellness metrics instantly. These cards provide users with a quick overview of the dataset's core indicators without requiring additional interaction.



We also apply percentage formatting and proper decimal formatting to ensure the values are readable and professionally presented.

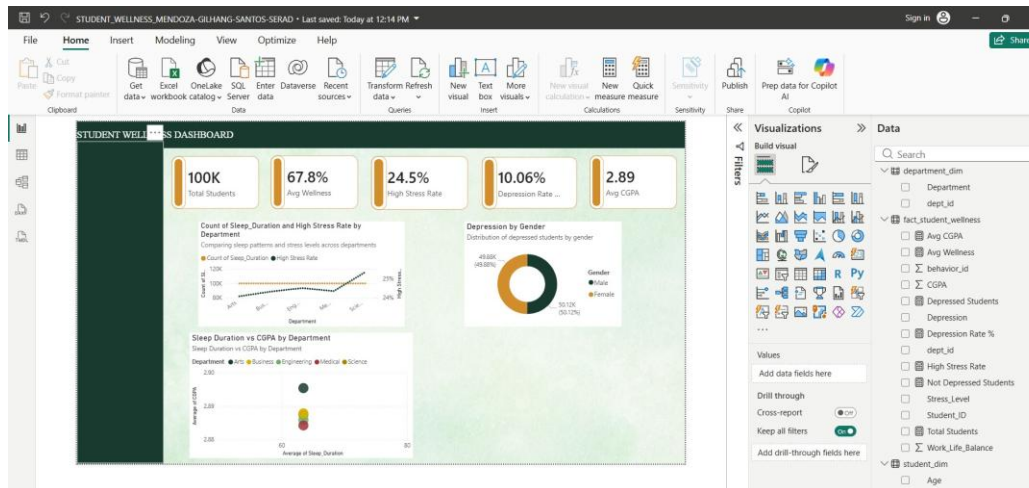
After setting up the KPI cards, we create our first analytical visualization titled:

For this visual, we use a **Line and Clustered Column Chart** to compare sleep duration counts alongside stress rate patterns across different departments. This visualization helps identify departments that may exhibit heavier wellness pressure or unhealthy sleep behaviors

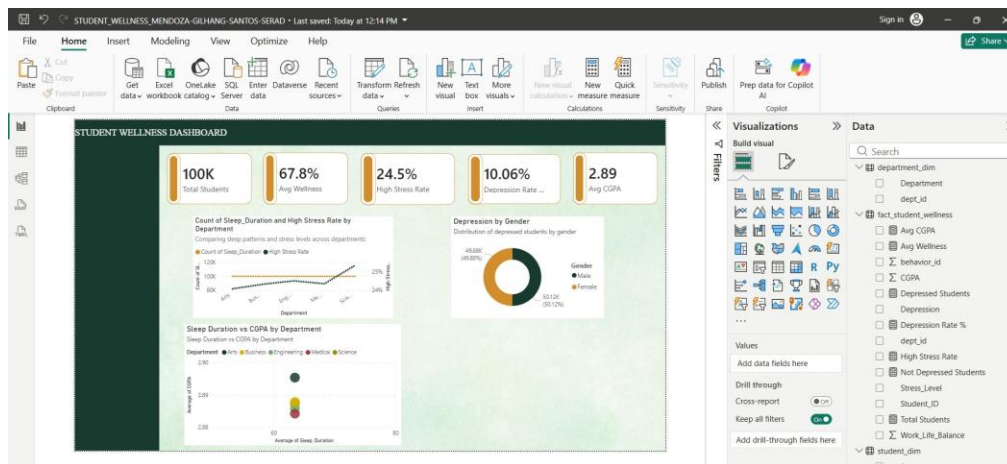


We also add a subtitle to improve readability and explain the purpose of the chart:

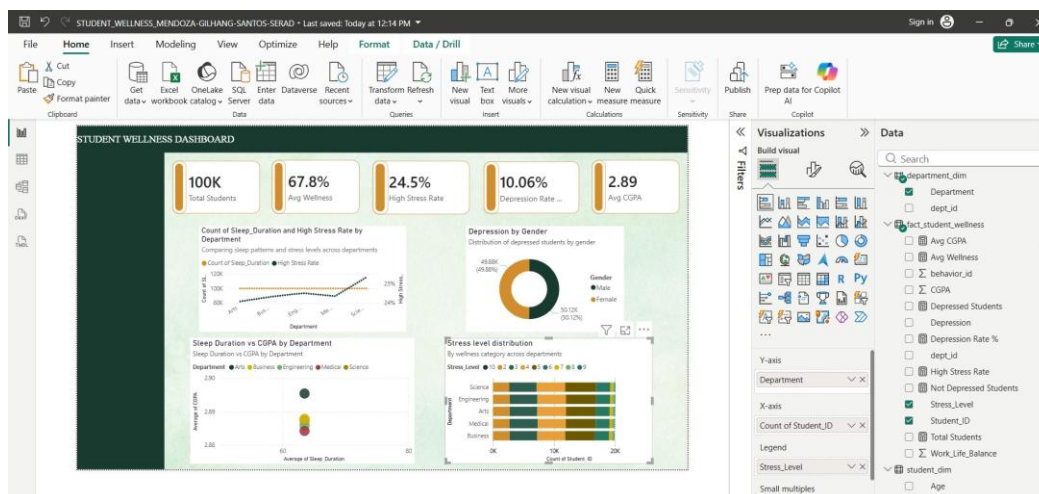
This chart analyzes the relationship between students' average sleep duration and academic performance (CGPA) while grouping the data points according to academic departments. Each colored bubble represents a department category, allowing viewers to identify patterns or correlations between sleep habits and educational performance.



We then proceed to build a Donut Chart titled:



This visualization displays the proportion of depressed students categorized by gender. Using this chart, users can quickly analyze the distribution of depression prevalence between male and female students.



This visualization displays the distribution of stress levels among different academic departments. By using stacked categories, users can compare stress concentration patterns between departments and identify which groups may experience greater wellness challenges.

To improve dashboard interactivity and user-driven analysis, we add slicers positioned vertically on the left side of the dashboard. These slicers allow users to dynamically filter all visuals simultaneously based on selected demographic and wellness categories.

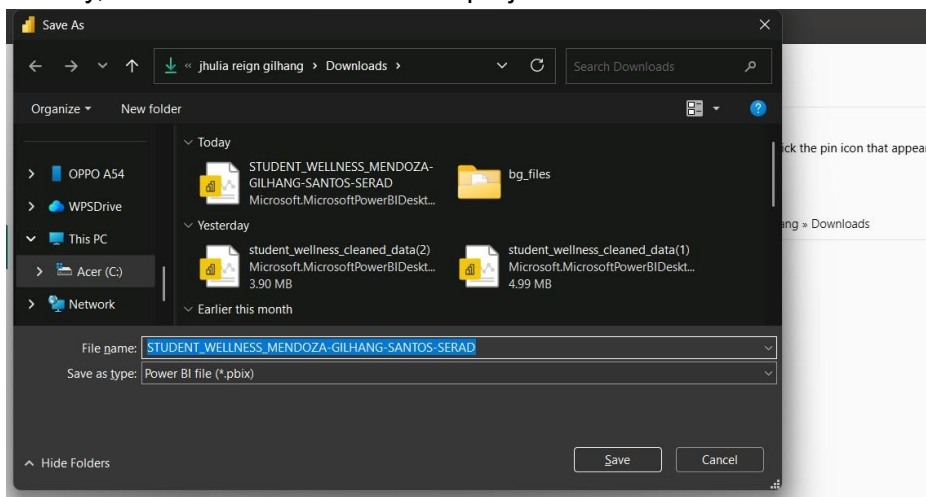


These interactive filters help users perform more focused exploratory analysis depending on their selected conditions.

After all visuals are completed, we perform final formatting adjustments such as:

- Aligning visuals consistently
- Adjusting font sizes and spacing
- Applying visual borders and rounded corners
- Ensuring chart titles and subtitles are readable
- Standardizing color consistency across all visuals

Finally, after validating that all visuals, slicers, measures, and relationships function correctly, we save the final Power BI project file as:



The dashboard is now fully interactive and capable of presenting analytical insights regarding:

- student wellness balance,
- depression prevalence,
- stress distribution,
- sleep behavior,
- and academic performance.

This concludes the Data Visualization and Dashboard Development phase of our Student Wellness Analytics Project.